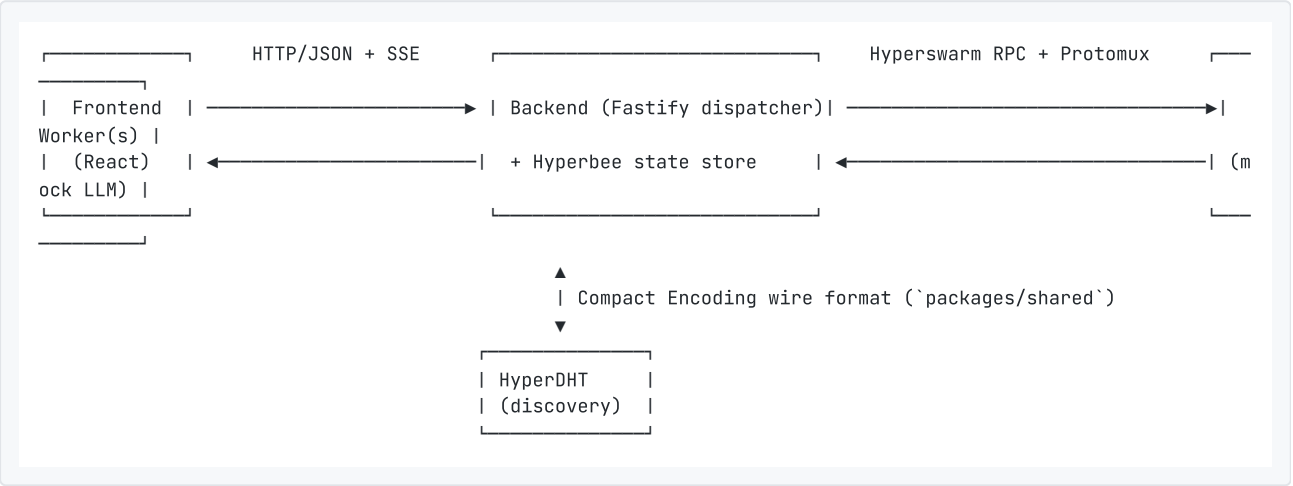
Cosmic GPU Cloud — Architecture

> *Living document. It will be appended to as each subsequent step lands. The intent is for the reviewer to find every architectural decision, trade-off, and known gap in **one** place rather than discovering them through code comments.*

1. System Design

The platform is split into three runtime processes plus shared contracts:



- **Backend** (`apps/backend`) — Fastify HTTP API. Owns auth, rate-limiting, request validation, dispatch decisions, and SSE bridging. Persists state in Hyperbee on Hypercore behind a `KvStore` interface (see `apps/backend/src/state/`).
- **Worker** (`apps/worker`) — Joins the Hyperswarm topic, exposes RPC methods (`inference.run` , `inference.stream` , `worker.health`), generates synthetic tokens. Will be wired in Step 5.
- **Frontend** (`apps/frontend`) — React/Vite dashboard for keys, playground, and usage. Will be wired in Steps 9–10.
- **Shared** (`packages/shared`) — TypeBox HTTP DTOs, Compact Encoding RPC schemas, constants, error codes. The single source of truth for cross-process contracts.

2. Distributed Approach

2.1 Discovery (Hyperswarm)

All nodes (backend + workers) join a shared 32-byte topic, configured via `P2P_DISCOVERY_TOPIC` . Every node announces with `client: true, server: true` , forming a complete-graph mesh for ≤10 peers. NAT traversal is automatic via HyperDHT.

For local Compose runs we expose an optional `P2P_BOOTSTRAP` env var (comma-separated `host:port` list). When empty, nodes fall back to the public Holepunch DHT bootstrap nodes. When set, nodes bootstrap against a deterministic local seed — useful for offline demos or networks that block the public DHT.

2.2 Identity

Every node persists a 32-byte ed25519 **seed** at `NODE_KEYPAIR_PATH` . On boot, the seed is loaded and a real `{ publicKey: Buffer(32), secretKey: Buffer(64) }` keypair is derived via `HyperDHT.keyPair(seed)` (libsodium `crypto_sign_seed_keypair`). The keypair is the same across restarts, so the worker registry's `workers/<pubkey>` rows remain stable, and Step 5 can pass the keypair directly to Hyperswarm. Public keys logged or shown in the UI are truncated to 8 hex chars + `…` .

2.3 Transport Split (RPC vs streaming)

`@hyperswarm/rpc` is a request/response RPC layer; it does not natively support streamed responses. We deliberately split the transport:

- `@hyperswarm/rpc` — `inference.run` (non-streaming completions) and `worker.health` (heartbeat).
- **Protomux channel** over the encrypted Hyperswarm secret stream — `inference.stream` (token-by-token streaming). Each chunk is a Compact-Encoded `InferenceResponseChunk`; the channel terminates with an `InferenceResponseFinal`. Cancellation is propagated by closing the channel; the worker observes the close and aborts its generator.

Both transports run on top of the same Noise-encrypted Hyperswarm session. The split is documented in `.cursor/rules/p2p-architecture.mdc` so the rule and this document agree.

2.4 State (Hyperbee on Hypercore, with HyperDB awareness)

The `KvStore` interface (`apps/backend/src/state/types.ts`) abstracts over two adapters:

- **In-memory** — used in tests and ephemeral dev (`STATE_BACKEND=memory`). Map-backed, deep-cloned values.
- **Hyperbee** — used in production (`STATE_BACKEND=hyperbee`). Single Hyperbee tree on a single Hypercore on disk, JSON-encoded values.

Typed accessors on top of `KvStore` (`api-keys`, `workers`, `jobs`, `usage`) keep the rest of the code backend-agnostic. Hypercore's signed Merkle trees provide integrity verification for free; sparse replication is the Hyperbee default and would activate automatically if we wired multi-peer state replication (see §6).

HyperDB vs Hyperbee — why we chose Hyperbee. The brief explicitly names `hyperdb` as the distributed K/V store. We evaluated the current package (`hyperdb@6.x`, March 2026) against our needs:

- HyperDB 6.x is a **schema-first document database** with named collections, indexes, and a build-time codegen step (`hyperdb/builder` + `hyperschema`). The runtime API is collection-scoped (`db.insert(collection, doc)`, `db.get(collection, queryObject)`, `db.find(collection, range)`), and `db.flush()` is mandatory to persist a batch.
- Internally, the P2P-aware `HyperDB.bee(core, definition)` engine wraps a Hypercore directly — it inherits the same append-only, signed-merkle-tree, sparse-replication semantics that Hyperbee provides. From a *distributed-systems* perspective the two share the same substrate; HyperDB's value-add is the schema + indexing layer above it.
- Our state shape is generic: string keys (`api-keys/<id>`, `workers/<pubkey>`, `jobs/<id>`, `usage/<keyId>/<windowStart>`) mapping to JSON values. We do not benefit from collection-typed records or index-driven queries today. Forcing a schema on top would mean shipping generated codegen artefacts for negligible gain.
- The `KvStore` interface keeps a future migration to a properly-typed HyperDB layout (one collection per resource, indexed by id and createdAt) feasible the day we need typed queries.

A runnable HyperDB awareness demo exercises `hyperdb@6.x` end-to-end (open Hypercore → `HyperDB.bee` → insert / get / find / delete / flush). Run it with `pnpm --filter @cosmic/backend demo:hyperdb`. It is intentionally *not* wired into the backend runtime — it is documentation in runnable form, the evidence behind this trade-off.

2.5 Wire format (Compact Encoding)

All RPC payloads are Compact Encoded with codecs in `packages/shared/src/schemas/`. The schema policy is **append-only**: never reorder or remove fields. New fields are added at the end and made optional so older writers still produce valid wire frames. Floats (e.g. `temperature`, `topP`) are encoded as fixed-point integers (×1000) so the wire stays integer-only; this rounds to 3-decimal precision (see Trade-offs §4).

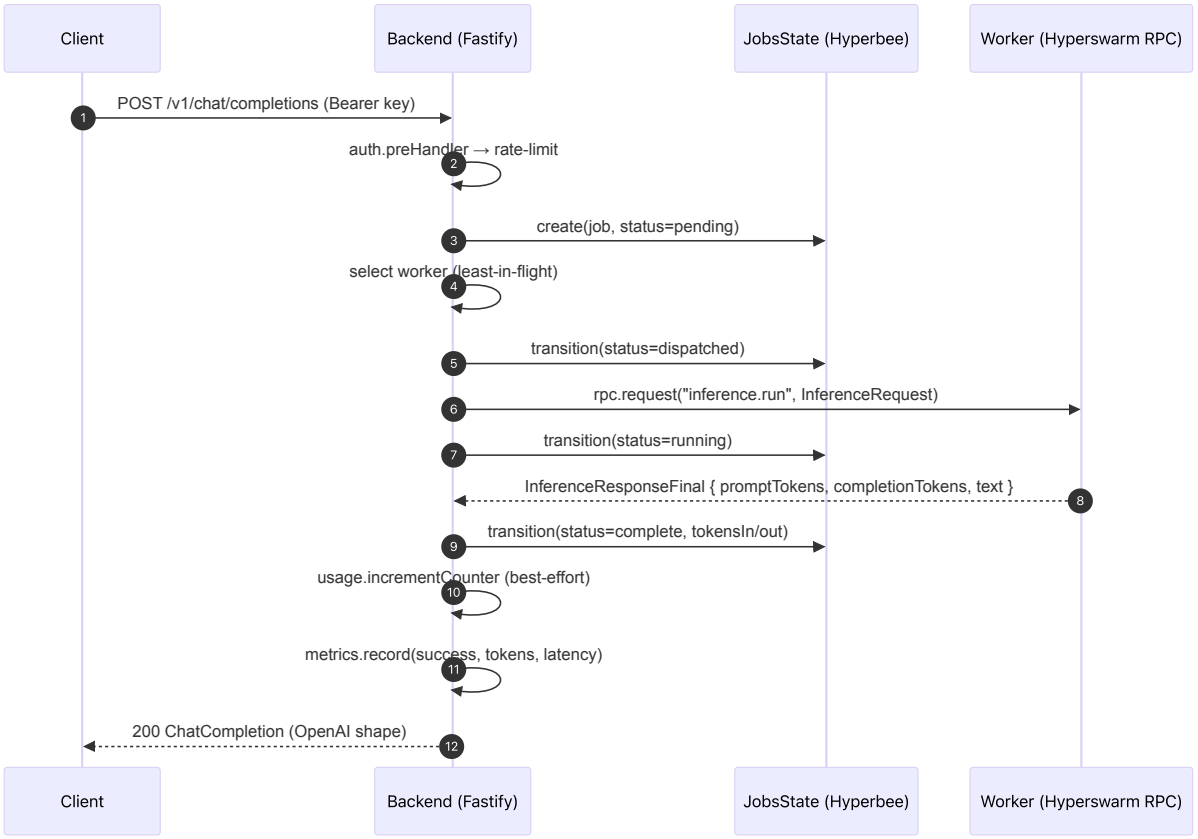
2.6 Workload distribution

The backend acts as a dispatcher (Step 6). Worker selection is least-in-flight with random tie-break, sourced from the local `WorkersState` accessor. Job state transitions are persisted in `JobsState` (`pending → dispatched → running → complete | failed`). A worker that misses a heartbeat for the configured TTL is soft-removed and any in-flight job is re-dispatched to another worker (non-streaming path) or surfaced to the client as an error event (streaming path).

RPC client + streaming transport share the swarm's HyperDHT instance. Both the `@hyperswarm/rpc` client and the Protomux streaming transport reuse `swarmHandle.swarm.dht` for outbound `dht.connect(workerPubKey)` lookups instead of bootstrapping their own DHT. The swarm has already populated its routing table via `swarm.join(topic)`, so reusing it makes RPC requests resolve immediately. Sharing is safe here because the backend never calls

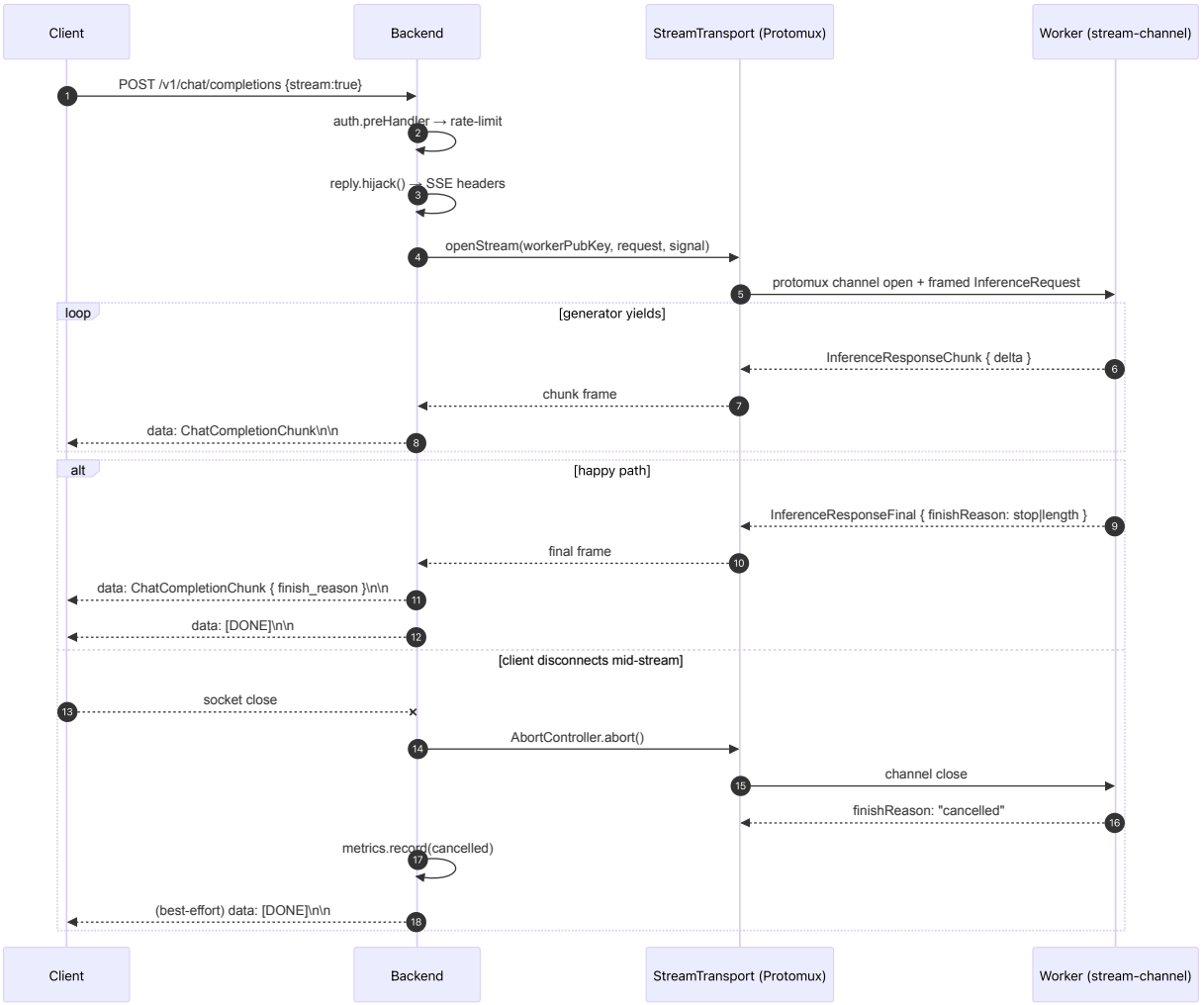
`dht.createServer().listen(keyPair)` (only outbound connects), so the `KEYPAIR_ALREADY_USED` constraint that applies to listeners doesn't apply to clients. The worker side keeps its own DHT for `rpc.createServer()` because that path **does** register a keypair on the DHT.

2.6.1 Non-streaming inference sequence



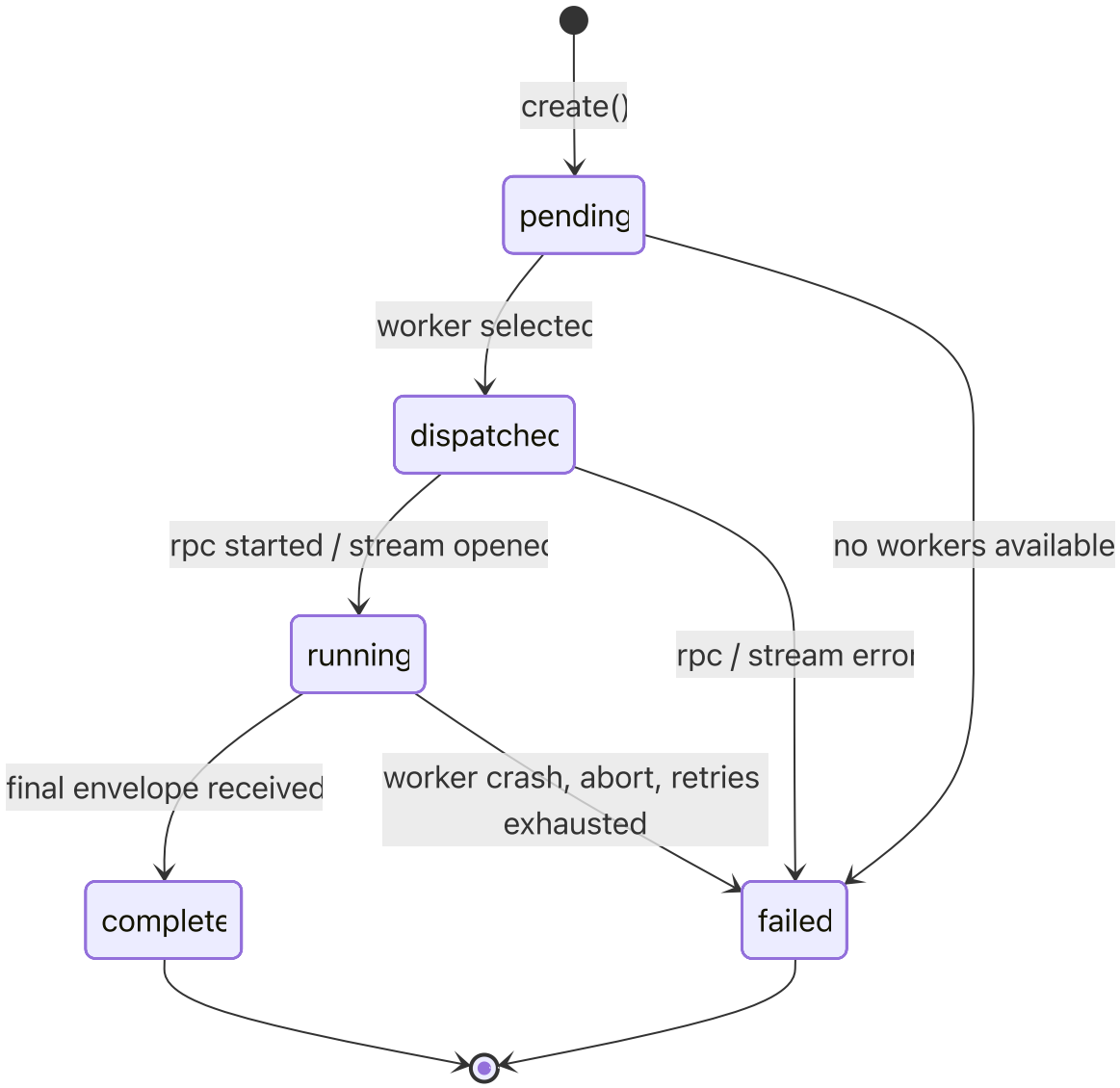
2.6.2 Streaming inference sequence (with client-disconnect cancellation)

Streaming uses a Protomux substream (`cosmic/inference-stream/v1`) over the encrypted Hyperswarm secret stream (§2.3). Cancellation propagates by closing the channel.



2.6.3 Job lifecycle state machine

`JobsState.transition(...)` (`apps/backend/src/state/jobs.state.ts`) enforces the following state machine.



3. Failure Scenarios and Resilience

Scenario	Response
All workers offline	Backend returns `503 NoWorkersAvailable` with `Retry-After`; the request is not queued in the MVP.
Worker disappears mid-stream	Backend terminates the SSE connection with a final `data: {error: WorkerTimeout}` event. No silent reconnection.
Client disconnects mid-stream	Backend's route hooks `request.raw.on('close')` → `AbortController.abort()`; the dispatcher's `dispatchStream` finally-block closes the Protomux channel; the worker's generator observes the close and emits `finishReason: 'cancelled'`. Job transitions to `failed` and `inference.failed` is logged.
Worker disappears mid non-stream	Backend re-dispatches once to another worker; on second failure, returns `503 WorkerTimeout`.
Topic mismatch / no peers found	Backend logs `warn` after N seconds with no peers and reports `p2p.connected: false` on `/health`.

Hyperbee write conflict	Last-write-wins with monotonic ISO timestamps on every record. Documented as a single-writer simplification.
Rate limit exceeded	429 envelope with `Retry-After`; per-key bucket.
Invalid bearer	`401 ApiKeyInvalid` envelope; revoked keys yield `403 ApiKeyRevoked`.

4. Trade-offs and Deliberate Divergences

These are intentional choices that differ from the original implementation plan or the most common production pattern. Each one is recorded so the reviewer doesn't have to spelunk through code comments.

- **Plain Node.js, not Pear Runtime / Bare.** The original implementation plan recommends Pear; we chose plain Node.js because Pear modules are explicitly described as potentially "unstable" and a 2-day window punishes ecosystem risk. Pear migration is captured as future work in `.cursor/rules/pear-runtime.mdc`.
- **Hyperbee on Hypercore, not HyperDB.** `hyperdb@6.x` is a schema-first document database with codegen-driven definitions; our generic key-string → JSON-value workload doesn't benefit from the schema layer. We use Hyperbee directly (the engine HyperDB itself sits on for its `bee` backend) and keep a future HyperDB swap feasible behind the `KvStore` interface. Detailed reasoning + a runnable HyperDB awareness demo: see § 2.4 and `apps/backend/scripts/hyperdb-demo/`.
- **Single-writer Hyperbee, not multi-peer replication.** The MVP holds state in one Hyperbee instance on the backend. Multi-writer state via `corestore` and per-worker Hypercore feeds is captured under § 6 Future Improvements.
- **Public API key management endpoints** (`POST/GET/DELETE /v1/api-keys` are not Bearer-protected). Without this, the first key creation is impossible without an out-of-band admin token. Production hardening (admin token, mTLS, or signed-installer flow) is captured under § 6.
- **In-memory rate limiter.** `@fastify/rate-limit` v10 with the default in-memory store means rate counters reset on restart and are not shared across multiple backend replicas. For a single-instance demo this is fine. For production, swap the store to Redis or a Hyperbee-backed sliding window (counter would live at `usage/<keyId>/<windowStart>` which we already model).
- **Mock LLM, not a real model.** Workers stream synthetic tokens. Real model serving is explicitly out of scope per the task brief.
- **Synchronous in-process error mapping.** A single Fastify error handler maps `DomainError` / validation / 429 / unknown to the shared `ApiErrorBody` envelope. We deliberately avoid spreading error formatting across handlers.
- **Fixed-point float encoding (×1000).** Inference parameters such as `temperature` and `topP` are encoded over RPC as `cuint` after multiplying by 1000. This rounds to 3-decimal precision, which is well within the OpenAI contract's tolerance and keeps the wire integer-only.
- **Mark & Sweep GC, Fork-ID truncate handling, and signed RPC results are deferred.** All three are captured in `.cursor/rules/p2p-architecture.mdc` and § 6 below. We never call `core.truncate`, so Fork-ID drift cannot occur in the MVP by construction.
- **Study guide note: "encrypted with private key, decrypted with public key".** That phrasing in the study guide describes a digital signature, not encryption. Hyperswarm sessions use **Noise** (an asymmetric handshake that derives symmetric session keys); confidentiality is symmetric, authenticity is asymmetric. Calling this out so reviewers don't think we adopted a wrong mental model.

4.1 Local Compose details

`docker-compose.yml` ships **self-contained by default**: a small `dht` service runs an in-Compose HyperDHT bootstrap node, and every backend / worker container sets `P2P_BOOTSTRAP=dht:49737`. This avoids Docker bridge-NAT hairpinning issues (see below) and works offline.

```
docker compose up --build             # default: in-Compose DHT bootstrap
P2P_BOOTSTRAP= docker compose up      # opt-out: public Holepunch DHT
```

Why self-contained by default — a Docker bridge-NAT note. When every node lives on the same Docker bridge (the typical laptop demo), each container's HyperDHT announce records the **host's** external NAT address, not the container's bridge-internal IP. Looking each other up through the public DHT therefore returns external addresses that the bridge can't reach back unless the host router supports NAT hairpinning — which most consumer setups don't. The result is a silent failure: every node announces successfully, no errors are logged, but `swarm.on('connection')` never fires because the UDP packets never arrive. Running an in-Compose `dht` service keeps announcements on bridge-internal IPs (`172.x.x.x`), which the bridge happily routes between containers. The opt-out (`P2P_BOOTSTRAP=`) is preserved so the public-DHT path stays exercisable for multi-host deployments where each container has its own routable IP.

Named volumes (`backend-data`, `worker[1-3]-data`) persist Hyperbee/Hypercore state and node-identity seeds across container restarts so worker pubkeys stay stable.

4.2 Operating notes

A few container-runtime details that surfaced during the Compose hardening pass and are worth knowing if you fork the Dockerfiles:

- **`libatomic1` is required at runtime.** Holepunch's `rocksdb-native` (a transitive dep through `udx-native` → `dht-rpc` → `hyperdht`) needs `libatomic.so.1` at `dlopen` time on Linux. The `node:22-bookworm-slim` base image doesn't ship it, so the `runtime` stage in `docker/{backend,worker}.Dockerfile` installs `libatomic1` explicitly. Without it, the prebuild's `dlopen` fails and the `require-addon` resolver surfaces a misleading `MODULE_NOT_FOUND` for one of the fallback candidate paths instead of the actual `libatomic` error.
- **Containers run as the unprivileged `node` user.** Both Dockerfiles `mkdir -p /data && chown -R node:node /data` before declaring the `VOLUME ['/data']` so the named volume's mountpoint is owned by uid 1000. Without this, the runtime crashes with `EACCES: permission denied, mkdir '/data/state'` on the first Hyperbee write because Docker initialises a brand-new named volume as `root:root`. Note this only affects the **first** attach — if you've previously created a volume under a buggier image, you need `docker compose down -v` to drop it before the new image's ownership takes effect.
- **In-Compose `dht` service runs from `/app/apps/backend`.** The backend image uses pnpm's isolated node-linker, so `hyperdht` lives at `/app/apps/backend/node_modules/hyperdht` (not a root-hoisted location). The Compose service overrides `working_dir` so `node -e 'require("hyperdht")'` resolves correctly.

4.3 Telemetry

The backend exposes a small in-memory metrics surface:

- `GET /metrics` — public, opt-out from rate-limiting (so liveness scrapers cannot DoS themselves). Returns a JSON snapshot of inference counters (`success`, `failed`, `cancelled`), token totals (`prompt`, `completion`), latency percentiles (`avg`, `p50`, `p95`, `p99`), worker pool gauges (`connected`, `capacity`), and rate-limit rejection counters.
- `GET /v1/usage/me` — authenticated. Returns the caller's usage totals plus the most recent windows (capped at 24 by default) sourced from `UsageState`. Powers the dashboard poll.
- **In-memory `MetricsRegistry`** (`apps/backend/src/observability/metrics.ts`) tracks counters, a fixed-size ring buffer of latency samples (default 1000), and lazy worker gauges. The dispatcher records outcome / token / latency observations on every dispatch (streaming + non-streaming); the central error handler increments the rate-limit rejection counter for every 429 envelope.
- **Counters reset on restart.** Same trade-off as the in-memory rate-limit store. Production hardening (Prometheus exposition, persistent counters, multi-replica aggregation) is captured under § 6 Future Improvements.

5. Scaling

The current design is single-region and single-backend-replica. To scale:

- **More workers.** Just start more containers; they join the Hyperswarm topic and announce themselves. The backend's `WorkersState` registry refreshes from heartbeats. Effort: zero code, just `docker compose --scale worker=N`.
- **Multiple backend replicas.** Requires a shared rate-limit store (move from in-memory to Redis or Hyperbee) and shared state (multi-writer via `corestore` or HyperDB). Effort: medium.

- **Geographic distribution.** Hyperswarm + HyperDHT already supports cross-NAT, multi-region peering. The bottleneck shifts to the Hyperbee state store; a per-region replica with conflict resolution would be the natural next step.
- **High-throughput inference.** Bypass JSON encoding at the worker and stream raw tensors directly via Hypercore feeds (sparse replication makes this efficient). Use one Hypercore feed per job.

6. Future Improvements

These are deliberate "should-haves" or "could-haves" that are documented but not implemented in the 2-day window.

6.1 Final-pass items (test consolidation)

Gaps the test suite would benefit from before a production handoff. Functional behaviour is in place; these are explicit assertions on hard-to-reach edge paths:

- **Dispatcher failure-mode coverage.** End-to-end assertions on the one-retry path, second-failure → `503 workerTimeout`, and empty-registry → `503 noWorkersAvailable`. The dispatcher logic exists and the routes return the right envelopes manually; targeted unit tests would lock in the contract.
- **Streaming cancellation E2E.** A test that drives the full chain: client `AbortController.abort()` → dispatcher closes the Protomux channel → worker generator observes the close → emits `finishReason: 'cancelled'` → backend records `cancelled` in metrics. Today the pieces are individually unit-tested but no single test traverses the entire chain on a live channel.
- **Frontend smoke tests for loading / error / empty states.** The Playwright e2e suite covers the happy path on each page. A dedicated set of smokes for `loading` (TanStack Query suspense), `error` (typed `ApiError` envelope rendering), and `empty` (no API key / no workers / no usage windows) states would back up the `frontend-standards.mdc` requirement.

6.2 Resource cleanup audit (Step 11.5)

Long-lived resources that currently rely on garbage collection or implicit close-chains. Items to either explicitly close or document with a known-gap rationale:

- **`KvStore.close()` actually closes the Hypercore feed.** The Hyperbee adapter's `close()` only calls `bee.close()`; the underlying Hypercore tree + file handles are released by GC instead of an explicit `core.close()`. Move to explicit teardown so test harness restarts don't accumulate FDs.
- **Hyperswarm connection objects after a peer disconnect.** `WorkerRegistry` soft-removes the entry from its `peers` Map, but the underlying `NoiseSecretStream` is left to GC. An explicit `conn.destroy()` would release the kernel UDP socket immediately rather than waiting on Node's idle handle close.
- **Protomux stream-channel listener registry on the worker side.** `attachStreamChannelServer` opens one channel per peer; on peer departure the channel object stays referenced by Protomux's internal map until the next inbound frame triggers cleanup. Explicitly tear down on the swarm `'connection'` close event.
- **Forgotten event listeners across the close-chain.** `request.raw.on('close')`, `reply.raw.on('error')`, channel `close` callbacks. Audit each consumer and remove listeners on the inverse path (`once`-style or explicit `removeListener` in `finally`) to satisfy the lint rule + avoid the rare leak that only shows up under load.

Output shape: explicit teardown calls + targeted assertions in `apps/backend/src/p2p/registry.test.ts` and the worker stream-channel tests. Audit takes ~30–60 min when scheduled.

6.3 Hardening / production candidates

- **Authenticated `/v1/api-keys` management.** Gate behind an admin bearer token (rotated via a signed installer) or mTLS.
- **Distributed rate-limit store.** Replace `@fastify/rate-limit`'s default memory store with a Hyperbee-backed sliding window keyed at `usage/<keyId>/<windowStart>`.
- **Signed RPC results.** Worker signs the canonical `InferenceResponseFinal` bytes with its `secretKey`; backend verifies against the registry's `pubKey`. Adds non-repudiation on top of Noise's confidentiality.
- **Multi-writer state.** Wrap state in `corestore`; let workers write their own audit feeds; backend reads via sparse replication.

- **Mark & Sweep GC.** `core.startMarking()` → mark active blocks → `core.sweep()` to reclaim disk after revoked keys / stale jobs.
- **Fork-ID truncate handling.** When (if) we ever call `core.truncate`, propagate the new fork id to all consumers.
- **Audit feed.** Replace structured Pino `event` lines with a dedicated Hypercore feed so audit history is replicable.
- **Hyperswarm firewall callback.** Allow-list mode for production deployments where worker pubkeys are known ahead of time.
- **Heartbeat jitter.** Random ±20% jitter on `worker.health` ticks so all workers don't ping at the same instant.
- **Prometheus exposition.** Add a `/metrics?format=prom` branch that serialises `MetricsRegistry.snapshot()` in Prometheus text format so existing scrapers can ingest it without parsing JSON.
- **Persistent metrics + multi-replica aggregation.** Move counters into a Hyperbee-backed sliding window (or Redis) so `/metrics` survives restarts and aggregates across multiple backend replicas.
- **Pear Runtime migration.** Re-evaluate after the ecosystem stabilises; see `.cursor/rules/pear-runtime.mdc`.

7. Review Checklist

For the reviewer / take-home grader:

- Auth happy path, missing/invalid/revoked branches: see `apps/backend/src/plugins/auth.plugin.test.ts`.
- Rate limit + 429 envelope: see `apps/backend/src/plugins/rate-limit.plugin.test.ts`.
- API key lifecycle (create → list → revoke): see `apps/backend/src/routes/api-keys.route.test.ts`.
- Wire format stability + edge cases: see `packages/shared/src/schemas/*.test.ts`.
- Identity persistence + recovery: see `apps/backend/src/state/identity.test.ts`.
- Quick start: see the README.
- Tracker / progress: see `docs/requirements-tracking.md`.